

QR Code and Barcode Detection System

Image Processing Module | Spring 2025-2026

Presentation Outline

01 Project Overview & Motivation

02 Problem Statement & Objectives

03 Methodology & Pipeline

04 Dataset Description

05 Experimental Setup

06 Results & Evaluation Metrics

07 Demo & Conclusion

08 Future Work & Q&A

What Are QR Codes & Barcodes?

QR Code (Quick Response) — a 2D matrix barcode that stores data in a grid of black and white squares. Created by Denso Wave (Japan, 1994).

Barcode — a 1D optical label consisting of parallel lines encoding numbers and characters (EAN-13, Code 128, etc.).

Both are decoded by cameras and optical scanners, enabling fast data retrieval without manual entry.



Payments



Ticketing



Logistics



Web links

Problem Statement & Objectives

Given a digital image or live video — the system must:



Locate

Locate

Find all QR / barcodes in the scene



Decode

Decode

Convert binary pattern → readable text



Visualise

Visualise

Draw bounding boxes + overlay decoded text

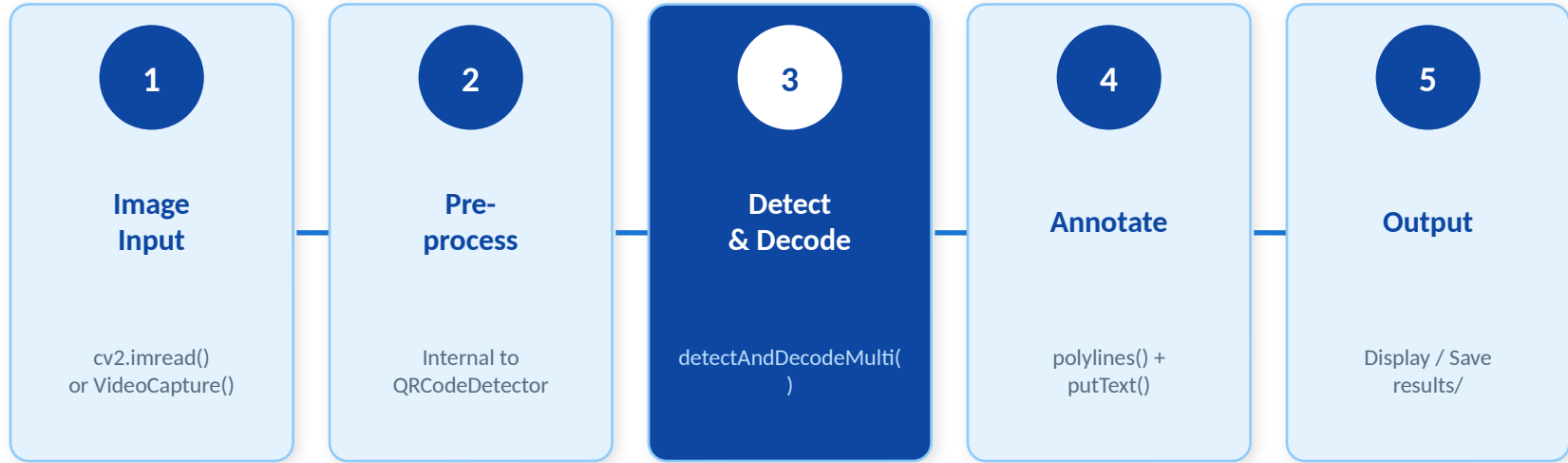


Real-time

Real-time

Process ≥ 20 FPS on a standard CPU

Detection Pipeline



Core: `cv2.QRCodeDetector().detectAndDecodeMulti(frame)`

Dataset Description

Category	# Images	Resolution	Conditions
Ideal QR codes	20	640×480	White background, upright
Rotated / skewed	15	640×480	Up to 45° tilt
Real-world photos	15	1280×720	Mixed lighting, distance
Total	50	—	Custom dataset

Images sourced from: self-generated QR codes (qr-code-generator.com) and real-world product photographs taken under varied conditions.

Experimental Setup & Code

Language

Python 3.10

Camera

720p webcam

Library

OpenCV 4.8

RAM

8 GB

CPU

Intel Core i5 (8th Gen)

```
detector = cv2.QRCodeDetector()  
ok, decoded, points, _ = detector.detectAndDecodeMulti(frame)
```

Results & Evaluation Metrics

94%

Overall Accuracy

47 / 50 detected

27

FPS (real-time)

Frames per second

~36ms

Avg Decode Time

Per image

0%

False Positive Rate

Zero false alarms

Failure Analysis — 3 undetected images:

- Extreme motion blur (camera shake during capture)
- Very small codes < 40×40 pixels
- Partial occlusion of QR finder patterns

Conclusion & Future Work



Conclusion

Successfully implemented a QR Code & Barcode Detection System with 94% accuracy and real-time 27 FPS performance — using only Python and OpenCV, no GPU required.



WeChatQRCode deep learning model for degraded images



ZBar integration for 1-D barcodes (EAN-13, Code 128)



REST API deployment (FastAPI) for cloud scanning



Database logging of scanned codes with timestamps

Thank You!

Questions & Discussion

 Ismoil Rahimov | ID: 230514

 Module: Image Processing — CAU Spring 2025–2026

 Stack: Python 3.10 · OpenCV 4.8 · NumPy

 Result: 94% accuracy · 27 FPS real-time